Programación Orientada a Objetos
2026 - 2do cuatrimestre - Miércoles tarde

Trabajo Práctico Obligatorio

Etapa 2: Codificación, puesta a punto y testeo

Propuesta B: Clínica veterinaria

Informe final y plan de pruebas

Integrantes:

Bustamante, Enrique

Franco, Tomás

Suárez, Elian

Repositorio: https://github.com/enriquebustamantebmw-hash/TPOVeterinariaPOO

1. Introducción

En esta etapa programamos en Java el sistema de la clínica veterinaria «Patitas» a partir del diagrama de clases final de la Etapa 1. La aplicación funciona por consola con un menú, guarda los datos en un archivo para que no se pierdan entre ejecuciones y tiene 58 pruebas unitarias sobre la lógica de negocio.

Las instrucciones para compilar, ejecutar y correr las pruebas están en el README del repositorio. En resumen: con Java 17 o superior y Maven se ejecuta `mvn package` y después `java -jar target/clinica-patitas.jar`. La primera vez se cargan datos de prueba (veterinarios, dueños, mascotas, stock, vacunas y turnos) para poder probar todo sin cargar nada a mano.

El código está organizado en estos paquetes:

Paquete	Qué tiene
clinica.modelo.personas	Persona, Dueno, Empleado, Veterinario, Recepcionista, Administrador, HorarioAtencion, PlanSalud, CargoPendiente
clinica.modelo.mascotas	Mascota y sus subclases Perro, Gato, Ave y Conejo, y los enum Especie, Sexo y Tamanio
clinica.modelo.turnos	Agenda, Turno, EstadoTurno, TipoAtencion y sus subclases ConsultaGeneral, Vacunacion y Cirugia
clinica.modelo.historia	HistoriaClinica, Consulta, Tratamiento, Vacuna (catálogo), VacunaAplicada, InsumoUtilizado, NotaRectificatoria, NuevaConsulta
clinica.modelo.stock	Insumo, Medicamento, Lote, Proveedor
clinica.modelo.facturacion	Factura, LineaFactura, NotaCredito, EstadoFactura, MedioPago, Efectivo, Debito, Credito
clinica.controladores	ControladorMascotas, ControladorTurnos, ControladorConsultas, ControladorFacturacion, ControladorAdministracion, DatosConsulta
clinica.servicios	ServicioStock, ServicioRecordatorios, Notificador, NotificadorArchivo, ObservadorStock, AlertaAdministrador
clinica.persistencia	Repositorio, RepositorioEnMemoria, Numerador, BaseDeDatos, ArchivoDatos, DatosIniciales
clinica.excepciones	ClinicaException y las excepciones propias de reglas de negocio
clinica.vista	MenuConsola y Consola (lectura de datos por teclado)

2. Diagrama de clases implementado

Al programar agregamos algunas clases y métodos que no estaban en el diagrama final de la Etapa 1 (lo explicamos en la sección 3), así que actualizamos el diagrama para que coincida con el código. Lo separamos en cuatro partes para que se pueda leer. Las fuentes (.mmd) y las imágenes están en /docs/uml.

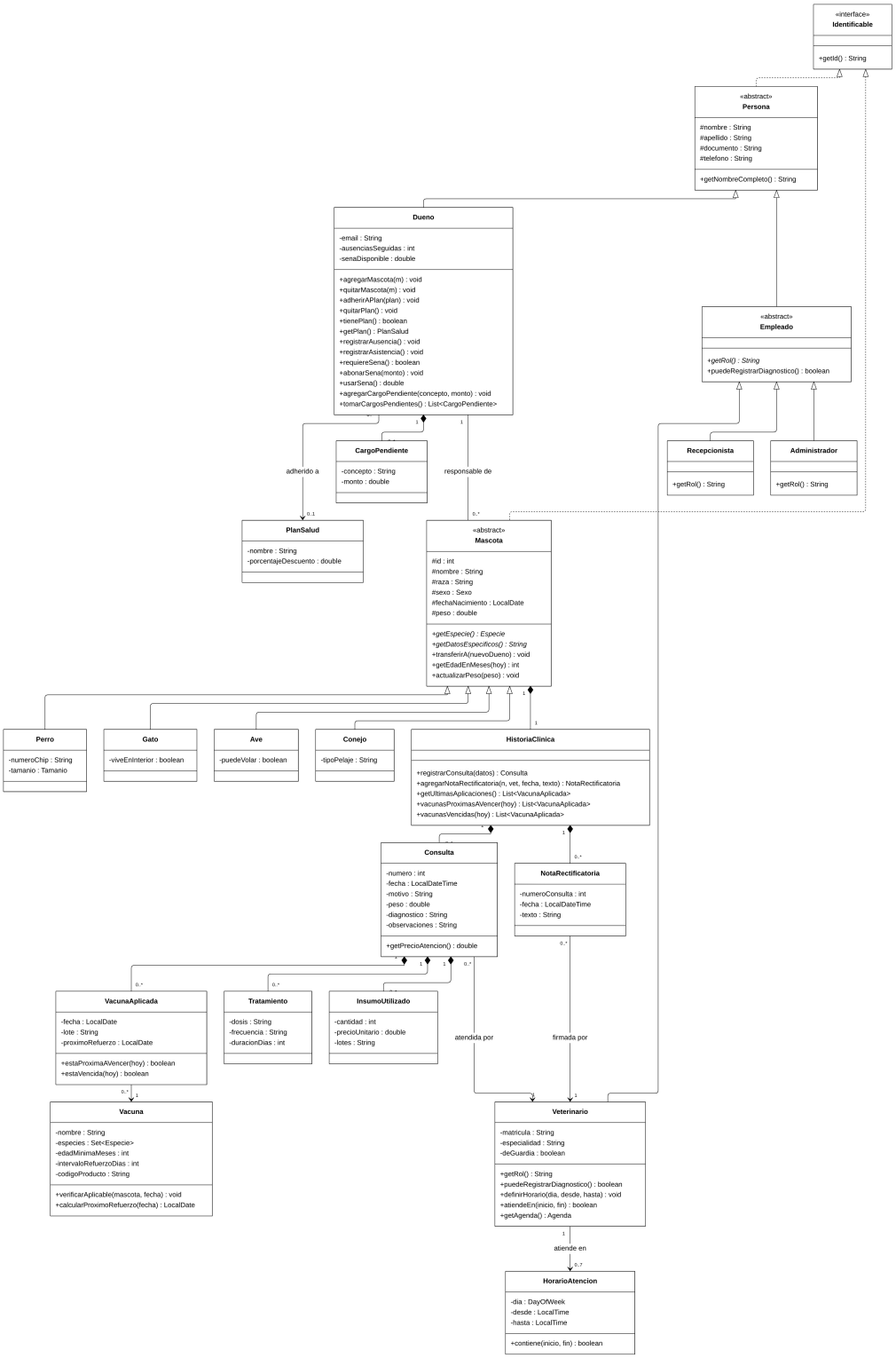


Figura 1: Personas, mascotas e historia clínica

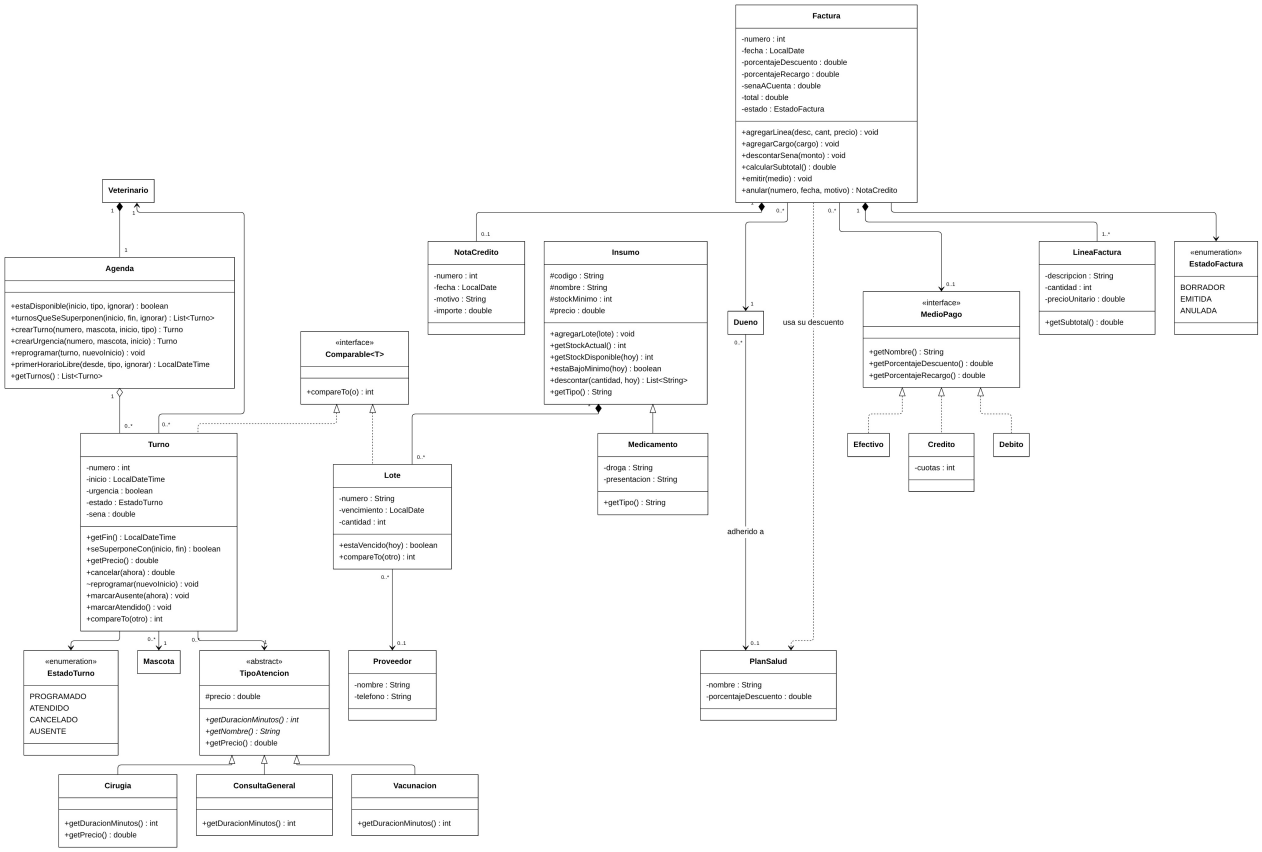


Figura 2: Turnos, stock y facturación (Veterinario, Mascota y Dueno están detallados en la figura 1)

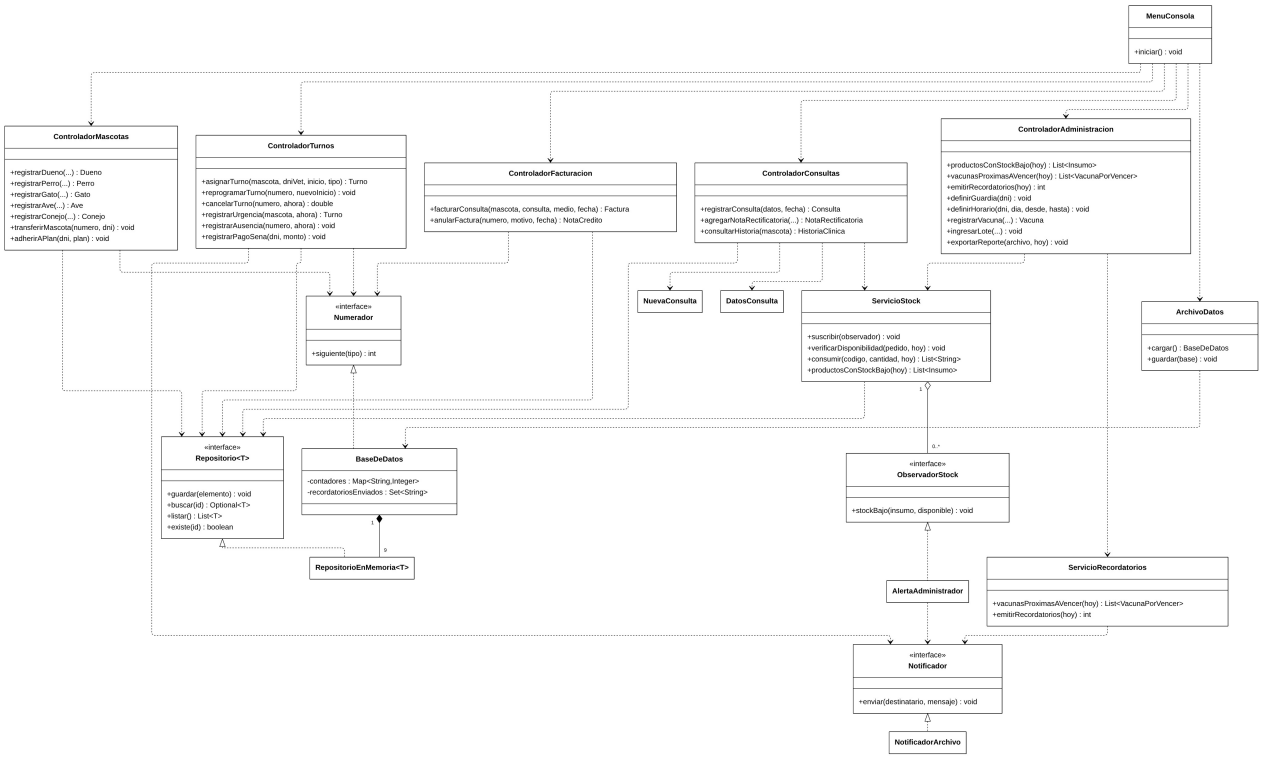


Figura 3: Vista, controladores, servicios y persistencia

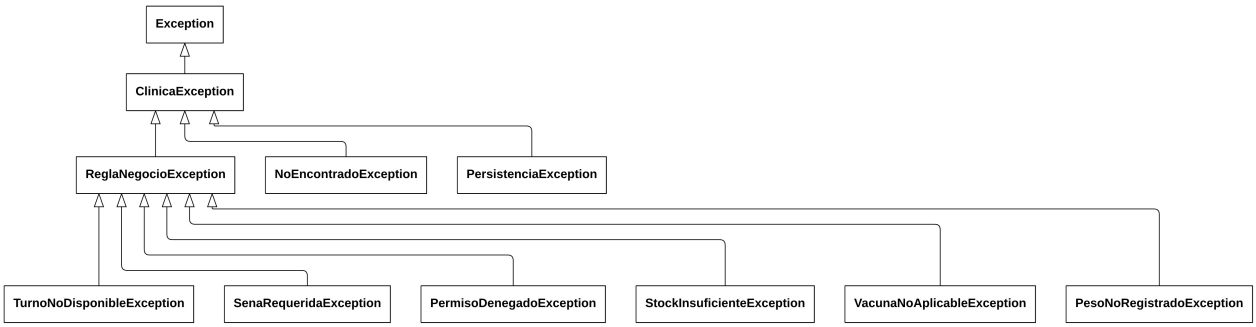


Figura 4: Excepciones propias

3. Cambios respecto del diseño de la Etapa 1

Antes de empezar a programar corregimos el documento de la Etapa 1 con la devolución y con el enunciado completo (reglas de vacunas, stock por lotes, facturación, historia clínica y notas de crédito). Por eso el código sigue casi en todo al diagrama final de esa entrega. Las diferencias que aparecieron al programar son estas:

N°	Qué cambió	Por qué
1	Agregamos el paquete vista con MenuConsola y Consola (y FinDeEntradaException).	El diagrama de la Etapa 1 no modelaba la interfaz. MenuConsola solo pide datos y llama a los controladores. Consola lee del teclado y vuelve a preguntar si el dato es inválido.
2	Agregamos la interfaz Identificable (getId()), que implementan todas las clases que se guardan.	Hace falta para que el repositorio genérico Repositorio<T> pueda buscar cualquier objeto por su id.
3	Agregamos DatosConsulta (lo que se carga en pantalla) y NuevaConsulta (los datos ya validados que recibe la historia clínica).	registrarConsulta() necesitaba muchos parámetros. Con estas dos clases el controlador valida todo antes de tocar la historia clínica, y HistoriaClinica sigue siendo la que crea la Consulta.
4	Agregamos BaseDeDatos, que junta todos los repositorios, y la interfaz Numerador para los números correlativos. ArchivoDatos guarda y lee la BaseDeDatos completa.	Para guardar todo en un solo archivo con serialización sin perder las referencias entre objetos (Veterinario, Agenda y Turno), y para que los números de turnos, mascotas y facturas sigan desde donde quedaron.
5	En lugar de NotificadorEmail y NotificadorWhatsApp implementamos NotificadorArchivo, que muestra el aviso por pantalla y lo guarda en datos/notificaciones.txt.	No tenemos un servidor de mail. Como los controladores dependen de la interfaz Notificador, el cambio no tocó ninguna otra clase.
6	Factura tiene agregarCargo() y getCargosIncluidos(). Al anular una factura, ControladorFacturacion devuelve esos cargos al dueño como pendientes.	Lo encontramos al ejecutar el plan de pruebas: al anular una factura se perdía el cargo por cancelación tardía que tenía.
7	ControladorTurnos controla la disponibilidad antes de pedir el número del turno.	También salió del plan de pruebas: cada intento fallido de asignar un turno gastaba un número.
8	Agregamos DatosIniciales, que carga los datos de prueba la primera vez que se ejecuta.	Para poder probar el sistema y hacer la demostración sin cargar todo a mano.

Supuestos de esta etapa

Los supuestos de negocio son los de la sección 2.4 del documento de la Etapa 1. En esta etapa solo agregamos dos:

- Para probar las reglas que dependen de la hora (cancelaciones, ausencias y urgencias), el menú deja ingresar la fecha y hora del momento. Si se deja vacío, usa la actual.
- La primera vez que se ejecuta se cargan datos de prueba con fechas calculadas a partir de ese día. Si existe datos/clinica.dat, se usan los datos guardados.

4. Principios y patrones en el código

Principio o patrón	Dónde se ve en el código
Responsabilidad única (SRP)	Cada controlador atiende un grupo de casos de uso. ServicioStock solo maneja el stock, ServicioRecordatorios los recordatorios, NotificadorArchivo los avisos y ArchivoDatos la lectura y escritura del archivo. MenuConsola solo pide datos y muestra resultados.
Abierto/Cerrado (OCP)	Para agregar un tipo de atención, un medio de pago, una especie o un observador de stock alcanza con una clase nueva que extienda TipoAtencion, implemente MedioPago, extienda Mascota o implemente ObservadorStock. Factura, Agenda y ServicioStock no cambian.
Inversión de dependencias (DIP)	Los controladores reciben por constructor Repositorio<T>, Numerador y Notificador (interfaces). Las implementaciones concretas se crean solo en Main. En las pruebas usamos un Notificador falso hecho con una lambda que guarda los avisos en una lista.
Experto en información	Agenda.estaDisponible() y primerHorarioLibre(), Turno.seSuperponeCon() y cancelar(), HorarioAtencion.contiene(), Dueno.requiereSena(), Insumo.descontar() y estaBajoMinimo(), Vacuna.verificarAplicable(), VacunaAplicada.estaProximaAVencer(), Factura.emitir().
Creador	Agenda crea los Turno (el constructor de Turno no es público). HistoriaClinica crea las Consulta y cada Consulta crea sus VacunaAplicada. Factura crea sus LineaFactura y su NotaCredito.
Controlador	ControladorMascotas (CU01 a CU03 y CU15), ControladorTurnos (CU04 a CU08), ControladorConsultas (CU09, CU10 y CU16), ControladorFacturacion (CU12 y CU17) y ControladorAdministracion (CU11, CU13, CU14, CU18 y CU19).
Polimorfismo	TipoAtencion.getDuracionMinutos() en cada subclase y Cirugia.getPrecio(). Mascota.getEspecie() y getDatosEspecificos() en cada especie. Los porcentajes de cada MedioPago. Empleado.puedeRegistrarDiagnostico(), que solo Veterinario redefine. Insumo.getTipo() en Medicamento.

5. Requisitos técnicos

Requisito	Cómo lo cumplimos
Lenguaje y paquetes	Java 17, proyecto Maven, organizado en los paquetes de la sección 1.
Herencia y polimorfismo	Jerarquías Persona → Empleado → Veterinario/Recepcionista/Administrador, Mascota → Perro/Gato/Ave/Conejo, TipoAtencion → ConsultaGeneral/Vacunacion/Cirugia, Insumo → Medicamento y las excepciones. Clases abstractas (Persona, Empleado, Mascota, TipoAtencion) e interfaces (MedioPago, Notificador, ObservadorStock, Repositorio, Numerador, Identificable).
Encapsulamiento	Atributos privados, atributos protected en las clases base (Persona, Mascota, Insumo, TipoAtencion), constructores con validaciones, constructores y métodos de paquete donde solo debe usarlos otra clase del paquete (Turno, Turno.reprogramar(), LineaFactura, VacunaAplicada) y listas devueltas sin poder modificarse.
Colecciones	List, Map (EnumMap de horarios, LinkedHashMap en los repositorios), Set (EnumSet de especies y recordatorios enviados). Orden natural con Comparable en Turno (por fecha) y Lote (por vencimiento). Orden con Comparator en mascotas por nombre, dueños por apellido, productos con stock bajo y vacunas por vencer por fecha.
Excepciones	Excepciones propias lanzadas con throw y declaradas con throws (TurnoNoDisponibleException, SenaRequeridaException, PermisoDenegadoException, StockInsuficienteException, VacunaNoAplicableException, PesoNoRegistradoException, etc.). try, catch y finally en ArchivoDatos, en ControladorAdministracion.exportarReporte() y en MenuConsola.iniciar(), que guarda los datos en el finally.
Entrada y salida	Los datos se guardan en datos/clinica.dat (ObjectOutputStream) y se leen al iniciar. Los avisos se agregan a datos/notificaciones.txt y el reporte del administrador se escribe en datos/reporte.txt.
Interfaz	Menú por consola con submenús, que vuelve a pedir los datos inválidos y muestra los errores sin cortar el programa.
Pruebas	58 pruebas unitarias con JUnit 5 (sección 6).

6. Pruebas unitarias

Las pruebas están en /test y se corren con `mvn test`. Usan un escenario con datos fijos (clase Escenario) y una fecha fija (lunes 9/11/2026), así no dependen del día en que se ejecutan.

Clase de prueba	Qué prueba	Cant.
TurnoTest	Duración por tipo, horario del veterinario, superposición, cancelación con y sin cargo, ausencias y seña, orden de los turnos	12
ControladorTurnosTest	Asignar y avisar, horario ocupado, seña requerida, cargo por cancelación, urgencia que desplaza turnos, reprogramación	7
ControladorConsultasTest	Permiso de la recepcionista, registro en la historia, peso para medicamentos, edad mínima de vacunas, lote que vence primero, stock insuficiente sin cambios, alerta de stock, turno atendido, historia no modificable, nota rectificatoria	11
FacturaTest	Efectivo, débito, crédito, plan contra efectivo, plan con crédito, seña, factura emitida no modificable, nota de crédito, factura vacía	9
ControladorFacturacionTest	Líneas de la factura, no facturar dos veces, cargo de cancelación en la factura, cargo que vuelve a quedar pendiente al anular	4
StockYVacunasTest	Lote que vence primero, lote vencido, stock mínimo, recordatorio a los 7 días y vencida a los 30, especie de la vacuna	5
MascotaYDuenoTest	Transferencia que conserva la historia, varias mascotas por dueño, datos por especie, permiso por rol	4
ControladorAdministracionTest	Alta en el catálogo de vacunas, vacuna repetida o con producto inexistente, orden del reporte de stock bajo, un solo veterinario de guardia	4
ArchivoDatosTest	Guardar y volver a leer la base, archivo dañado	2

Resultado de la última ejecución: **Tests run: 58, Failures: 0, Errors: 0, Skipped: 0 - BUILD SUCCESS**.

7. Plan de pruebas funcionales

Ejecutamos el programa por consola con los datos de prueba iniciales el 05/11/2026. Los turnos de prueba quedaron para el viernes 06/11/2026: con Laura Gómez el #1 de Firulais a las 10:00, el #2 de Michi a las 10:30 y el #3 de Kiwi a las 11:00, y con Martín Pérez el #4, la cirugía de Copo a las 9:00. La entrada completa que usamos y la salida que dio el programa están en /docs/pruebas.

ID	Caso de uso	Datos de entrada	Resultado esperado	Resultado obtenido	OK
PF01	CU01 Registrar dueño	Lucía Fernández, DNI 42555666	Se registra el dueño	«Dueño registrado: Lucía Fernández (DNI 42555666)»	Sí
PF02	CU01 Registrar dueño	DNI repetido 40111222	Error por DNI repetido	«ERROR: Ya hay un dueño registrado con el DNI 40111222»	Sí
PF03	CU02 Registrar mascota	Gata Luna de Lucía, vive en interior	Se registra con sus datos de especie	«Mascota registrada: #6 Luna (Gato...) - Vive en interior»	Sí
PF04	CU04 Asignar turno	Kiwi, vacunación con Gómez el 06/11 10:15 (ocupado por el turno #1)	Error de superposición y propuesta de horario libre	Error con «Primer horario libre: 06/11/2026 10:45». Se acepta y se crea el turno #5 a las 10:45	Sí
PF05	CU04 Asignar turno	Copo con Pérez el sábado 07/11 10:00 (los sábados no atiende)	Error por estar fuera de horario	«...no está disponible. Primer horario libre: 09/11/2026 09:00»	Sí
PF06	CU04 Asignar turno	Toby (dueño con 3 ausencias seguidas, sin seña)	No se da el turno	«ERROR: Pedro Sosa tiene 3 ausencias seguidas y debe abonar una seña...»	Sí
PF07	CU04 con seña	Pago de seña de $5.000 y turno para Toby el 06/11 12:00	Se da el turno con la seña	Turno #6 asignado. «Se tomó la seña de $5,000.00 para este turno»	Sí
PF08	CU06 Cancelar turno	Turno #3 (06/11 11:00) cancelado el 05/11 20:00	Sin cargo (más de 2 horas antes)	«Turno cancelado sin cargo.»	Sí
PF09	CU06 Cancelar turno	Turno #2 (vacunación 06/11 10:30) cancelado el 06/11 09:00	Cargo del 20% ($2.000)	«Turno cancelado con cargo de $2,000.00» y aviso a la dueña	Sí
PF10	CU07 Registrar urgencia	Michi el 06/11 10:00 (horario del turno #1 de Firulais)	La urgencia toma el horario y el turno #1 se reprograma con aviso	Urgencia #7 a las 10:00. Aviso: «el turno de Firulais se reprogramó para el 06/11/2026 11:00»	Sí
PF11	CU08 Registrar ausencia	Turno #4 (06/11 9:00) registrado el 06/11 12:00	Turno ausente y una ausencia para el dueño	«Juan Martínez lleva 1 ausencia seguida.»	Sí
PF12	CU09 Registrar consulta	La recepcionista (DNI 35123456) carga un diagnóstico	Se rechaza	«ERROR: Sofía Ruiz (Recepcionista) no puede registrar diagnósticos ni tratamientos»	Sí
PF13	CU09 Registrar consulta	Tratamiento con AMOX y peso 0	Se rechaza por falta de peso	«ERROR: No se puede indicar un medicamento si no se registró el peso...»	Sí
PF14	CU09 Registrar consulta	Antirrábica a Toby (1 mes; mínimo 3 meses)	Se rechaza por edad	«ERROR: Toby tiene 1 mes y la edad mínima para Antirrábica es 3 meses»	Sí
PF15	CU09 Registrar consulta	Firulais, turno #1, peso 19, AMOX, antirrábica y 1 jeringa	Consulta registrada, vacuna del lote que vence primero sin usar el vencido, alerta de jeringas en el mínimo	Consulta 2 registrada. Vacuna del lote L-ANT-02 (L-ANT-01 está vencido), refuerzo 2027-11-05. Aviso: «Reponer Jeringa...: quedan 10 unidades (mínimo 10)»	Sí
PF16	CU10 Historia clínica	Mascota #1	Muestra las 2 consultas y el estado de la vacuna	Se listan las consultas 1 y 2, y «Vacuna Antirrábica: refuerzo el 2027-11-05 (al día)»	Sí
PF17	CU16 Nota rectificatoria	Sobre la consulta 2 de Firulais	Se agrega sin modificar la consulta	«Se agregó: Nota rectificatoria sobre la consulta 2...»	Sí
PF18	CU12 Facturar	Consulta 2 de Firulais en efectivo (la dueña tiene plan y un cargo de $2.000)	Atención + vacuna + jeringa + cargo = $35.300 con 15% (plan mayor que efectivo) = $30.005	Factura 1: subtotal $35,300.00, «Descuento: 15.0% (plan de salud)», TOTAL $30,005.00	Sí
PF19	CU12 Facturar	La misma consulta otra vez	Se rechaza	«ERROR: La consulta 2 de Firulais ya tiene una factura emitida»	Sí

PF20	CU17 Anular factura	Factura 1	Nota de crédito por el total	«Nota de crédito 1 ... anula la factura 1 - $30,005.00»	Sí
PF21	CU12 Facturar	Consulta 1 de Michi, crédito en 3 cuotas	$25.000 + el cargo de $2.000 (que volvió a quedar pendiente) = $27.000; menos 15% y más 8% = $24.786	Factura 2: subtotal $27,000.00, descuento 15%, recargo 8%, TOTAL $24,786.00	Sí
PF22	CU19 Reportes	Reporte del administrador	Copo (mixomatosis el 09/11). Firulais ya no, porque se vacunó en PF15	«2026-11-09 - Copo (#4) - Mixomatosis»	Sí
PF23	CU13 Recordatorios	Emitir dos veces seguidas	1 recordatorio la primera vez y 0 la segunda	«Recordatorios enviados: 1» y después «Recordatorios enviados: 0»	Sí
PF24	CU19 Reportes	Reporte del administrador	Jeringas y Meloxicam	«JER: 10 disponibles (mínimo 10)» y «MELOX: 3 disponibles (mínimo 3)»	Sí
PF25	CU03 Transferir mascota	Copo pasa a Lucía	Cambia de dueño y conserva la historia	«Dueño: Lucía Fernández» y «La historia clínica se conserva (1 consulta).»	Sí
PF26	CU19 Reportes	Opción exportar reporte	Se crea datos/reporte.txt con los tres reportes	Archivo generado con turnos por veterinario, stock bajo y vacunas por vencer	Sí
PF27	CU18 Catálogo de vacunas	Leucemia felina, solo gatos, 2 meses, refuerzo cada 365 días, producto V-TRI	Se agrega al catálogo	«Vacuna agregada: Leucemia felina - especies [Gato] - edad mínima 2 meses - refuerzo cada 365 días»	Sí
PF28	Persistencia	Salir y volver a abrir el programa	Se cargan los datos guardados	«Se cargaron los datos guardados.» Aparece Lucía con Luna y Copo, y Juan con 1 ausencia	Sí

Mientras armábamos este plan encontramos dos errores que corregimos antes de la entrega. Un intento fallido de turno consumía un número de turno, y al anular una factura se perdía el cargo por cancelación tardía que tenía. Para el segundo agregamos una prueba unitaria.

8. Dificultades encontradas

- **Reglas que no estaban en el modelo.** Al releer el enunciado completo vimos que la primera versión de la Etapa 1 no cubría varias reglas (lotes, catálogo de vacunas, descuentos fijos, nota de crédito, nota rectificatoria). Corregimos el diseño antes de programar, junto con las observaciones de la devolución.
- **La urgencia.** Fue la regla más difícil. Al buscar el nuevo horario del turno desplazado, la agenda lo encontraba ocupado por él mismo. Lo resolvimos con un parámetro para ignorar ese turno al controlar la disponibilidad.
- **Validar antes de guardar.** Si en una consulta fallaba una vacuna o faltaba stock de un insumo, quedaban descontados los insumos anteriores. Reordenamos el método para validar todo primero y confirmar después.
- **Pruebas con fechas.** Muchas reglas dependen del día y la hora. Para que las pruebas den siempre lo mismo usamos una fecha fija y pasamos el momento actual como parámetro en vez de usar now() adentro de las clases.
- **Guardar en archivo.** Con la serialización tuvimos que hacer Serializable todas las clases del modelo y cuidar que las referencias cruzadas (Veterinario, Agenda y Turno) se reconstruyeran bien al leer.
- **Acentos en la consola de Windows.** Se ven mal en cmd si no se cambia la página de códigos. Lo dejamos explicado en el README.

9. Distribución de tareas

Integrante	Tareas
Bustamante, Enrique	Turnos y agenda (Turno, Agenda, TipoAtencion), ControladorTurnos con urgencias y ausencias, persistencia en archivo, pruebas de turnos
Franco, Tomás	Mascotas, historia clínica y vacunas, stock por lotes, ControladorConsultas y ServicioStock, pruebas de consultas y stock
Suárez, Elian	Facturación y medios de pago, ControladorFacturacion y ControladorAdministracion, menú por consola, README y plan de pruebas funcionales

El diagrama actualizado y este informe los revisamos entre los tres.

10. Reflexión del grupo

Lo que más nos sirvió fue haber hecho el diseño antes de programar. Como ya teníamos separados los controladores, los servicios y las interfaces de repositorio y notificador, muchos cambios quedaron en una sola clase. Por ejemplo, el notificador por mail se reemplazó por uno que escribe en un archivo sin tocar los controladores, y los descuentos se resolvieron dentro de MedioPago y Factura.

También aprendimos que el diseño no queda cerrado en la Etapa 1. Al programar aparecen detalles que en el diagrama no se ven, como el orden en que se validan las cosas o qué pasa con un cargo cuando se anula una factura. Por eso fue importante probar los casos de uso completos y no solo cada clase por separado: los dos errores que encontramos aparecieron recién al ejecutar el plan de pruebas funcionales.

Por último, nos quedó claro que hay que leer el enunciado completo y con cuidado desde el principio. Varias de las correcciones de la primera entrega vinieron de reglas que estaban escritas y no habíamos tenido en cuenta.